Lab 6 and Lab 7 - Ski Resort Pass

Coverage analysis, Hypothesis fuzz testing, and input validation fixes

Student: Vinayak Yuvraj Kalal SRN: PES2UG25AM809
GitHub: kalalvinayak23 Verified environment: Python 3.12.14

Assigned repository: https://github.com/SETAPESU26/L6_L7_08_ski_resort_pass

Check	Observed result
Initial Lab 6 starter	47% line coverage; 44% combined branch-mode coverage
Final line test suite	14 passed; 59/59 executable statements covered (100%)
Final branch test suite	47 passed; 42/42 branch exits covered (100%); no partial branches
Original Lab 7 fuzz tests	3 failed; Hypothesis reduced each crash to the input None
Fixed Lab 7 suite	44 passed, including six property tests and regression cases

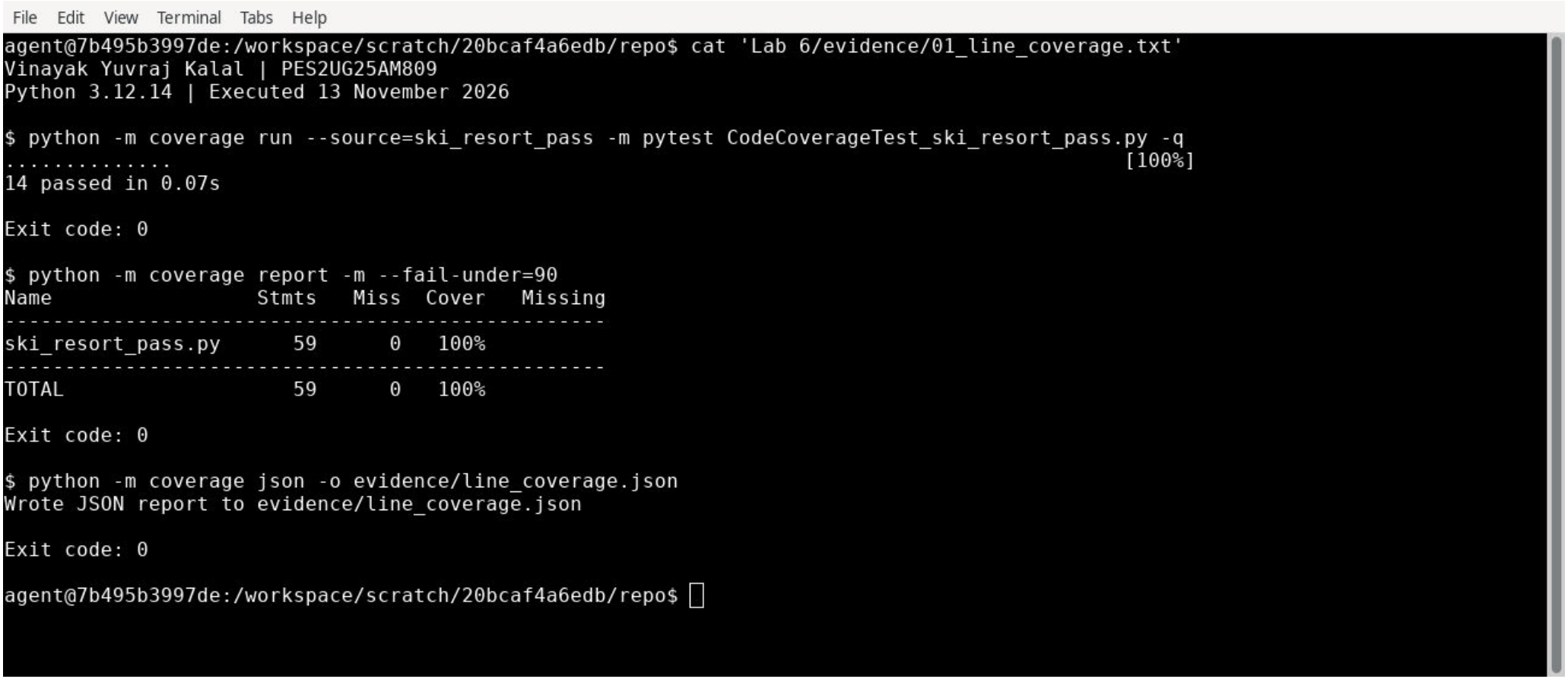
Fixes: validate string types, calendar dates, numeric structure and ranges; reject zero distance, NaN, infinity and overflow; handle unknown zone prefixes safely.

Behavior retained: the date status follows the starter's fixed year >= 2026 rule. Zone prices retain the BEG/INT/ADV/ALL prefix convention. Lab 6 application logic is unchanged.

Screenshots show recorded runs displayed in a real terminal. Full text logs and machine-readable coverage reports accompany this PDF. Coverage does not prove every possible input or path is correct.

Lab 6 - Line coverage

Required code coverage test file: CodeCoverageTest_ski_resort_pass.py



Result: 14 passing tests, 59 statements, zero missed statements, 100% line coverage. The 90% threshold passes.

Lab 6 - Branch coverage

Required branch coverage test file: BranchCoverageTest_ski_resort_pass.py

```
File  Edit  View  Terminal  Tabs  Help
agent@7b495b3997de:/workspace/scratch/20bcaf4a6edb/repo$ cat 'Lab 6/evidence/02_branch_coverage.txt'
Vinayak Yuvraj Kalal | PES2UG25AM809
Python 3.12.14 | Executed 13 November 2026

$ python -m coverage run --branch --source=ski_resort_pass -m pytest BranchCoverageTest_ski_resort_pass.py -q
.............................................                                                [100%]
47 passed in 0.12s

Exit code: 0

$ python -m coverage report -m --fail-under=90
Name                Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------
ski_resort_pass.py      59      0     42      0   100%
-----------------------------------------------------------------
TOTAL                   59      0     42      0   100%

Exit code: 0

$ python -m coverage json -o evidence/branch_coverage.json
Wrote JSON report to evidence/branch_coverage.json

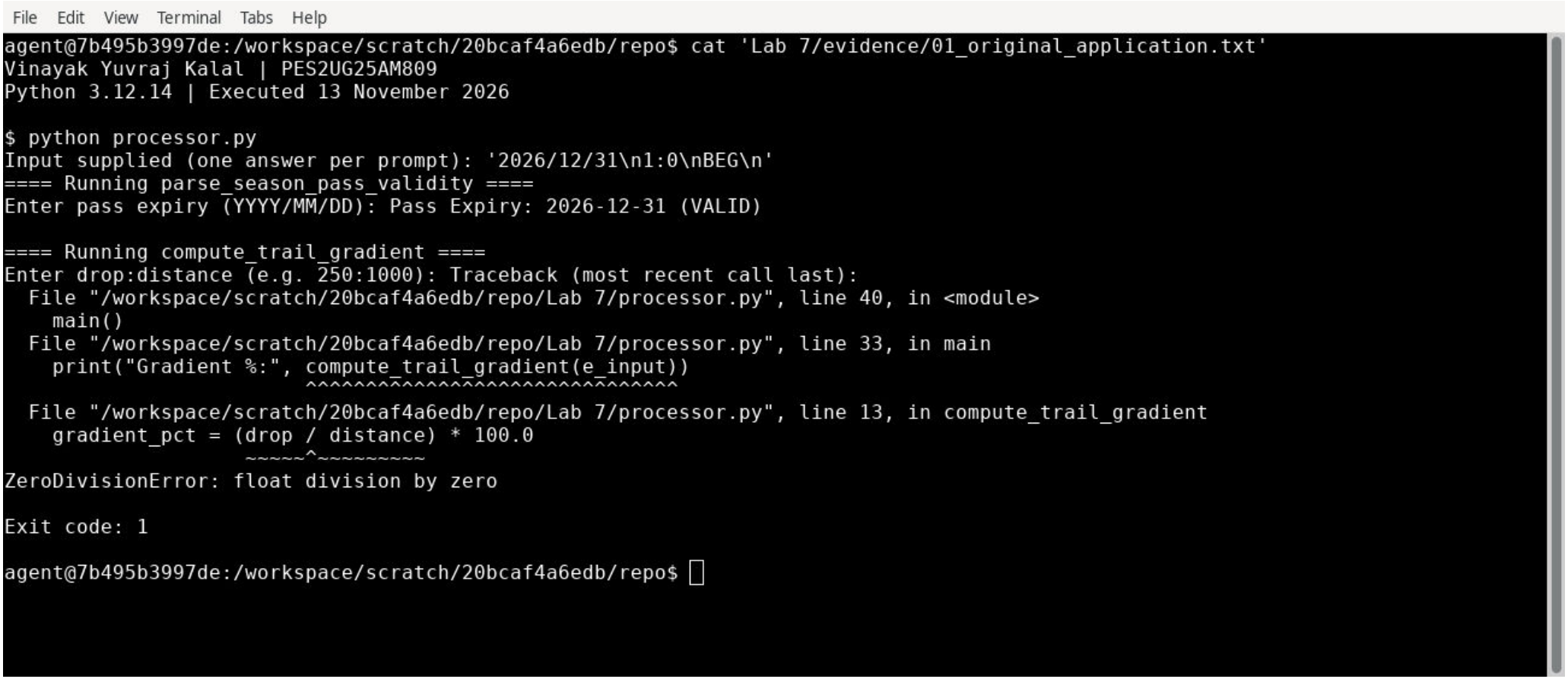
Exit code: 0

agent@7b495b3997de:/workspace/scratch/20bcaf4a6edb/repo$
```

Result: 47 passing tests, 42 branch exits, zero partially covered branches, 100% coverage. The JSON report independently records 42 covered and zero missing branch exits.

Lab 7 - Original application failure

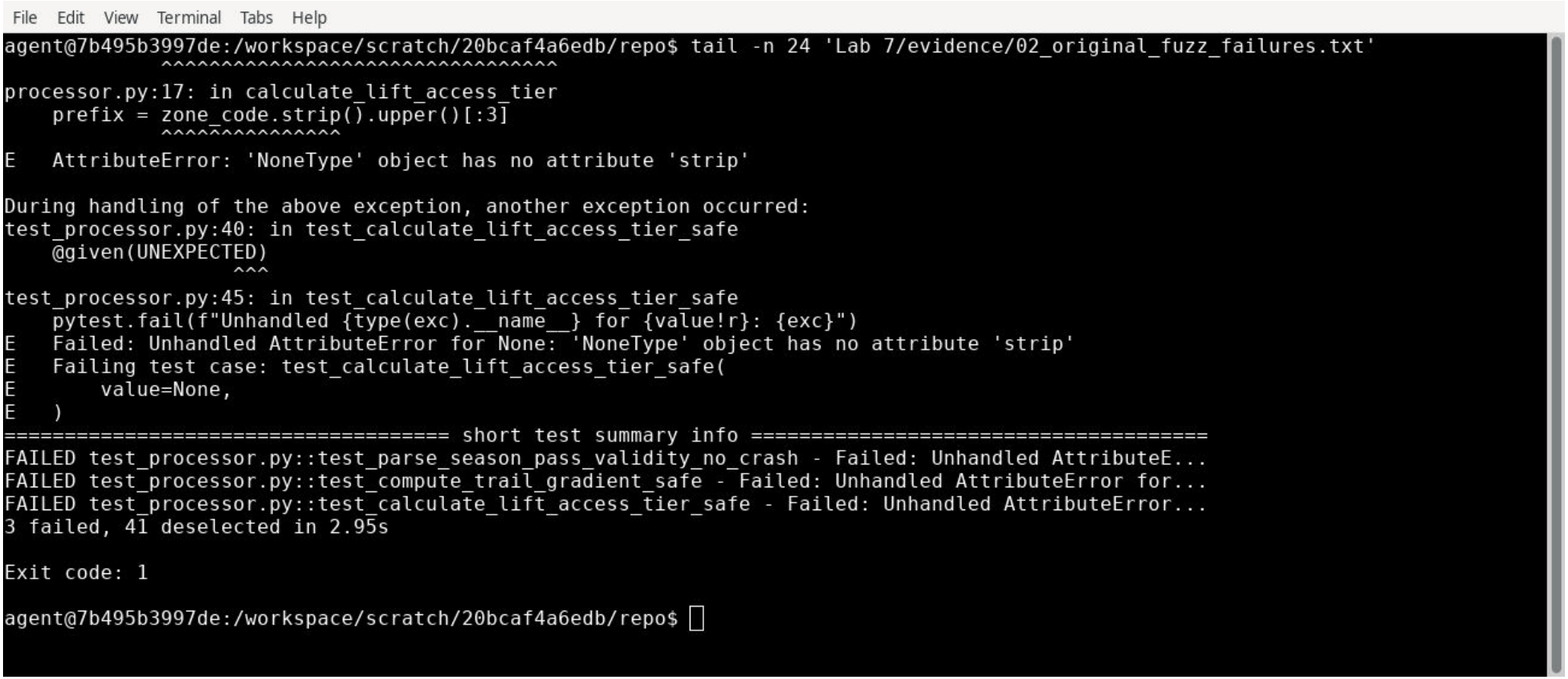
Before the fix: a zero distance crashes the application.



Inputs: 2026/12/31, 1:0, BEG. The gradient divides by zero. The application exits with code 1 before processing the zone code.

Lab 7 - Original fuzz-test failures

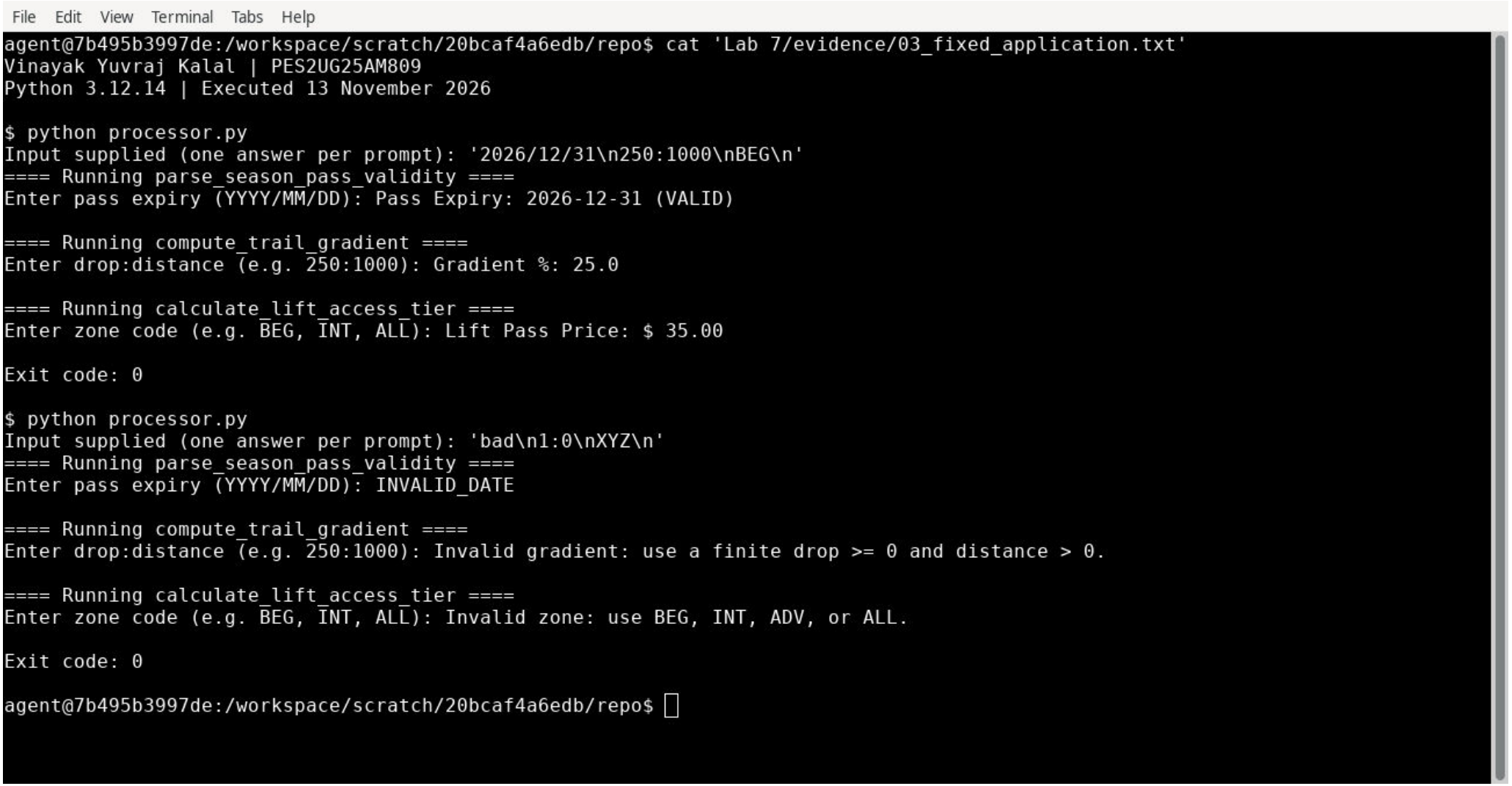
All three robustness properties fail against the unmodified starter.



The terminal displays the last 24 lines of the full failure log. Each failed property was automatically reduced by Hypothesis to None, causing AttributeError in string processing. The 41 deselected tests are additional checks excluded from this focused before-fix run.

Lab 7 - Fixed application output

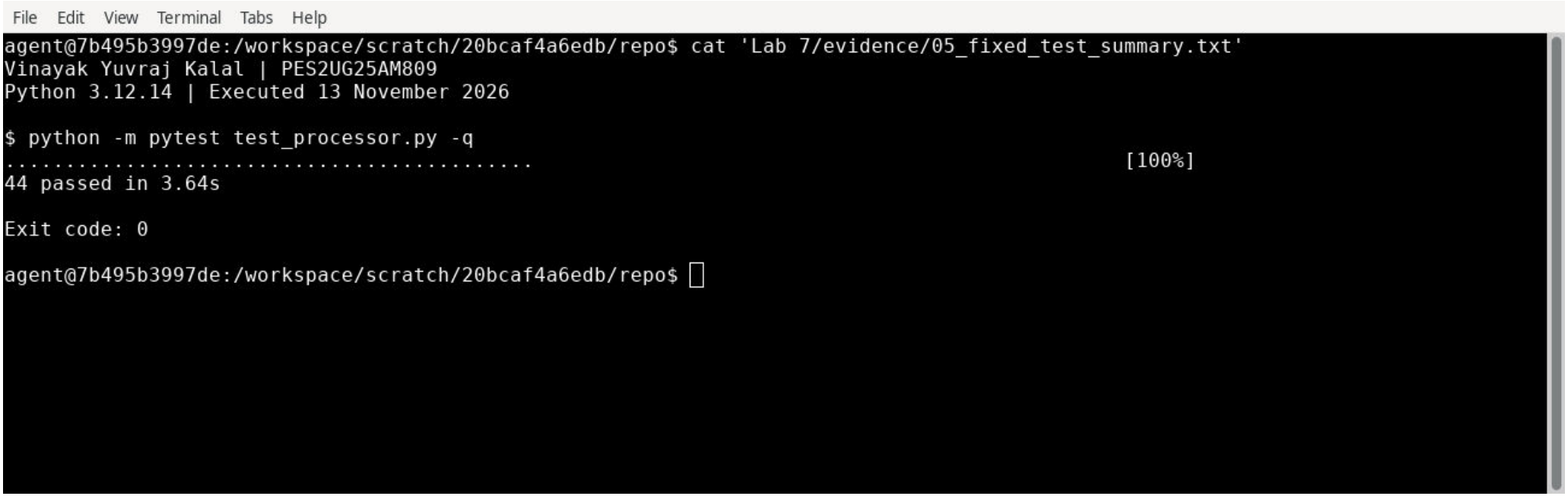
Valid inputs produce the expected results; invalid inputs return clear messages.



Both runs finish with exit code 0. Valid results: VALID, 25.0%, $35.00. Invalid date, zero distance and unknown zone inputs are handled without an uncaught exception.

Lab 7 - Fixed fuzz tests pass

All 44 property and regression tests pass against the fixed processor.py.



The full statistics are saved in evidence/04_fixed_fuzz_tests.txt.

Generated passing examples in the recorded run: 500 for each of the three robustness properties, 300 real dates, 500 valid gradients, and all 32 zone/case/whitespace combinations (2,332 generated passing examples in total). Ordinary regression cases additionally check impossible dates, leap years, negative values, nonfinite values, zone prices and CLI behavior.

Run python run_checks.py to verify the finished project. Add --replay-original to demonstrate the original failures in a temporary copy before running the fixed checks.